

# AI Solutions for Vessel Identification in SAR Images

Micol Zazzarini and Andrea Fiorani<sup>†</sup>

Contributing authors: [s1120529@studenti.univpm.it](mailto:s1120529@studenti.univpm.it);  
[s1116254@studenti.univpm.it](mailto:s1116254@studenti.univpm.it);

<sup>†</sup>These authors contributed equally to this work.

## Abstract

In this project, undertaken as part of our Computer Vision examination, we tackled the challenge of ship detection using Synthetic Aperture Radar (SAR) images. Our approach focused on the hyperparameter tuning of two YOLO (You Only Look Once) architectures: YOLOv8s, and YOLOv10s. Initially, we performed extensive hyperparameter optimization to enhance the performance of each model. Following this, we conducted an optimization phase, aiming to improve the efficiency of the models. The results demonstrate the efficacy of our tuned models in accurately detecting ships in SAR imagery, achieving good results in terms of Mean Average Precision, Recall, Precision, and Loss values. This work not only contributes to the field of maritime surveillance but also provides insights into the practical application of hyperparameter tuning and inference optimization in object detection tasks.

**Keywords:** YOLO, hyperparameter tuning, detection, SAR imagery

## 1 Introduction

Ship detection and classification using optical Remote Sensing (RS) images have significant applications in both military and civilian fields, drawing increasing interest. Convolutional Neural Networks (CNNs) have shown exceptional detection capabilities in dense environments, surpassing traditional methods. Since the 1990s, data from SAR satellites have become widely available, leading to extensive research on SAR ship detection.

Synthetic Aperture Radar (SAR) offers significant advantages over other sensors, such

as optical and infrared sensors, by producing high-resolution images both day and night and in all weather conditions. Recent advancements in spaceborne SAR imaging have significantly improved the quality and resolution of SAR images, making them crucial for maritime management and tracking, particularly for ship identification. Ship identification in high-resolution SAR (HR-SAR) images has been extensively studied due to the clear scattering features and geometric structure of ship targets. Despite this, challenges remain, including dense surroundings and issues like sidelobes and target defocusing. These images are crucial for applications like illegal fishing detection, oil spill identification, marine traffic management, and combating maritime piracy.

In particular, the goal of our work has been to improve the current state of the art in vessel identification using Synthetic Aperture Radar (SAR) imagery. Our project is part of a larger initiative developed by National Research Council of Italy (CNR), dedicated to monitoring marine resources, with a primary focus on enhancing the identification of suspicious vessels, particularly concerning Illegal, Unreported, and Unregulated (IUU) fishing activities. The goal is to provide a more precise and reliable tool to combat such activities, thereby contributing to the protection of marine ecosystems and the sustainability of global fishery resources.

Systems like Europe’s Copernicus program’s Sentinel-1 provide numerous SAR images for maritime applications. The Deep Learning approach has proven increasingly effective for SAR ship detection, making SAR one of the best instruments available for satellite-based ship identification.

## 2 State of Art

In the field of ship identification using SAR images [1], techniques can be divided into two categories: traditional algorithms and deep learning-based algorithms. Traditional approaches use handcrafted features and statistical models to distinguish between background and ships, such as Generalized Likelihood Ratio Test (GLRT) and Constant False Alarm Rate (CFAR) detectors [2]. Although these techniques have been successfully applied, they have limitations in terms of adaptability and rely heavily on statistical models of sea clutter. Furthermore, due to contaminated backgrounds, distribution parameters may be inaccurately approximated, leading to severe missed detections.

Hence, techniques based on deep learning have been developed, both at single-stage and two-stage levels. In single-stage methods, object detection is treated as a regression problem. Coordinates of bounding boxes and probabilities of pixel class membership in the image are generated. Commonly used are networks like YOLO (see [3] and [4] on bibliography), Single Shot MultiBox Detector (SSD), RetinaNet, CenterNet, HyperLiNet, fully deformable convolutional network (FD-Net), two-way convolution network, MW-ACGAN Network, ship detection and orientation estimation (SDOE), etc.

Two-stage techniques, on the other hand, create region proposals as potential bounding boxes and then design a classifier to categorize these bounding boxes. Subsequently,

bounding boxes are refined through post-processing, and duplicate identifications are removed. Common two-stage methods include Fast RCNN, Faster RCNN, Mask RCNN, dense attention pyramid networks (DAPN), quad feature pyramid network, multi-task learning-based object detector, etc.

Generally, single-stage algorithms are simpler and faster to train compared to two-stage object detection techniques, which are more accurate.

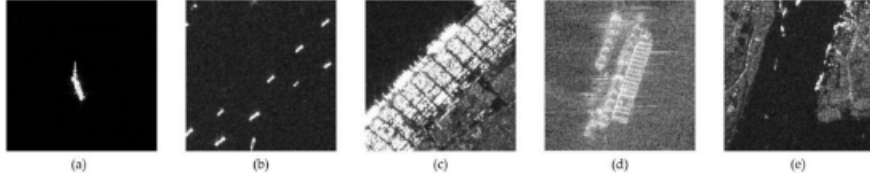
### 3 Materials and Methods

#### 3.1 Dataset

The dataset we used for our project is a *SAR Dataset of Ship Detection for Deep Learning under Complex Backgrounds* (see [5] on bibliography), created by SAR experts. We have chosen this dataset because, unlike others, it is open source, and from the experimental results conducted by the experts, emerged that object detectors achieve higher mean average precision on it and have higher generalization performance. It was created using 102 *Chinese Gaofen-3* images and 108 *Sentinel-1* images, and consists of 43,819 ship patches.

These ships have distinct scales and backgrounds, and they vary in terms of polarization, resolution, incidence angle, imaging mode, and background.

Ships appear as bright pixels mainly due to the double reflection of pulses emitted by the sensor. Additionally, the incidence angle influences the geometry of the ships, producing foreshortening, layover, and shadowing, and these factors pose significant challenges for ship detection. You can see on Figure 1 some examples of ship images.

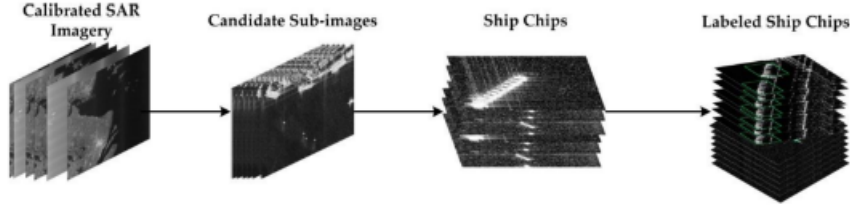


**Fig. 1:** Some examples of ship chips

The procedure used by the experts for the dataset construction is shown on Figure 2. First, all SAR images are converted to  $\sigma_0$  values. SAR images are created using radar systems mounted on satellites, which emit microwave signals towards the Earth and record the reflected signals. These images are based on the time delay and intensity of the radar signals bouncing off surfaces.  $\sigma_0$ , the *radar backscatter coefficient*, is a measure of surface reflectivity as seen by the radar, quantifying how much of the radar signal is reflected back to the sensor. This measure is crucial for interpreting SAR images. The raw SAR images from the Chinese Gaofen-3 and Sentinel-1 satellites contain radar signal intensities that need to be standardized. Converting these intensities to  $\sigma_0$  values normalizes the radar returns, allowing consistent comparison across different images and conditions, thus improving the accuracy of ship detection algorithms and facilitating the detection and comparison of objects across

various images and backgrounds.

Second, candidate subimages containing ships with sizes greater than 800 pixels are cropped. Third, sliding windows from these candidate subimages are used to acquire ship images 256×256 pixels in size. To enrich the backgrounds of the ships, 128 pixels are shifted over both columns and lines during the sliding window, leading to 50% overlap of adjacent ship images. To mark the ships position, these ship images are converted to gray images. Finally, they are labeled by SAR experts with LabelImg.



**Fig. 2:** Procedures for building the labeled dataset

Regarding the properties of the dataset, there are two main factors for SAR ship detection, including ship size and background influencing detection performance. Compared with those objects in the optical dataset, the ship size is relatively quite small. Therefore, the ships may be a pixel and miss the characteristics of ships on a feature map after subsampling several times in deep convolutional networks, thus making it difficult to distinguish ships. Since the targets on the backgrounds of the ships, such as buildings, harbor, noise, and islands may share similar backscattering mechanism to ships, thus leading to false alarms.

Typically, traditional methods use *land-ocean segmentation* to address this issue. However, this method limits the speed of ship detection and hinders automatic end-to-end ship detection. To overcome this, ship chips in this dataset include ships in *complex backgrounds*.

The shapes of ships can be inherently diverse and exhibit multiple scales. The same ship in images of different resolutions may have different sizes. Additionally, SAR images are influenced by their imaging mechanism. Geometric deformation caused by the incidence angle also affects the sizes of ships in SAR images. To address these variations and enhance robustness in ship detection, the dataset employs "*relative bounding box size*". This method standardizes and measures ship sizes relative to the chip (or region of interest) in which they are contained within the image. The larger the ship compared to the surrounding region, the greater the relative bounding box size. This approach helps compensate for scale and geometry variations of ships in SAR images, thereby improving detection accuracy and result consistency.

The dataset has been already tested on a series of architectures like *SSD*, *Faster R-CNN*, and *RetinaNet* (for more details, see reference [5] on bibliography). Considering the SAR characteristics, data augmentation is discarded. This is based on the assumption the ship chips after data augmentation may not fit with the SAR imaging

mechanism. The experimental results, shown on Figure 3, revealed that, although there

| Model            | Input Size (pixels) | mAP (%) | Training Time (minutes) |
|------------------|---------------------|---------|-------------------------|
| SSD-300          | $300 \times 300$    | 88.32   | 193.25                  |
| SSD-512          | $512 \times 512$    | 89.43   | 253.01                  |
| Faster R-CNN     | $600 \times 800$    | 88.26   | 531.4                   |
| RetinaNet        | $800 \times 800$    | 91.36   | 650.77                  |
| Modified SSD-300 | $300 \times 300$    | 88.26   | 127.89                  |
| Modified SSD-512 | $512 \times 512$    | 89.07   | 221.83                  |

**Fig. 3:** Results given by experiments conducted by SAR experts

is no land ocean segmetation, the models trained on these data can detect multiscale ships with low numbers of false positives under complex backgrounds.

### 3.2 Tested Architectures

Our experiments are conducted on the YOLO (You Only Look Once) architecture. It is an effective choice for the task of ship detection on SAR images for several reasons, such as temporal efficiency, effective handling of small-sized objects, robustness to lighting and background variations, and ease of implementation. Furthermore, we have also decided to focus primarily on YOLO, as it is the only architecture not tested by the SAR experts who created the dataset we chose for our experiments. We have observed that using YOLO, it is possible to achieve good accuracy values on the reference data, even with a significant increase in speed compared to other computationally heavier algorithms.

YOLO networks, starting from version YOLOv1 up to the more recent YOLOv8 and YOLOv10, are known in fact for their ability to perform real-time detections with a good balance between accuracy and speed. YOLOv5, in particular, has shown significant improvements over its predecessors thanks to a better network architecture and advanced optimization techniques, and YOLOv8 and YOLOv10 represent further evolutions of this framework, with incremental improvements in detection capabilities and increased computational efficiency. For this reason, based primarily on a study of the YOLOv5 architecture and of its applications on SAR images (see [3] reference), we decided to conduct our experiments on the last two architectures, as these represent improved versions of the first, aiming to further improve the results, in terms of efficiency and effectiveness

The integration of these advanced neural networks with SAR data has proven to be effective, leveraging the penetration capability and high spatial resolution of SAR images.

#### 3.2.1 YOLOv5

YOLOv5, introduced by Ultralytics, is one of the most widely used versions of YOLO due to its balance of speed and accuracy. It consists of several versions with essentially

the same structure but different depths and widths, and the network structure consists of the *backbone*, *neck*, and *head* (see on Figure 4, for more informations see reference [6] on bibliography).

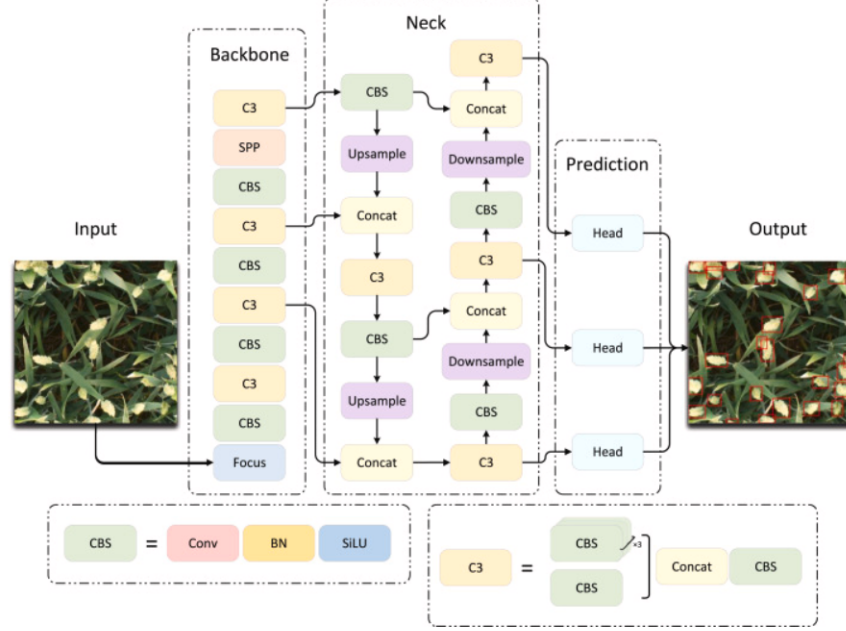


Fig. 4: YOLOv5 architecture

- **Backbone (CSPDarknet53):** The backbone is used to extract features of different scales of images, including low-level features extracted from shallow layers and high-level semantic features extracted from deep layers.

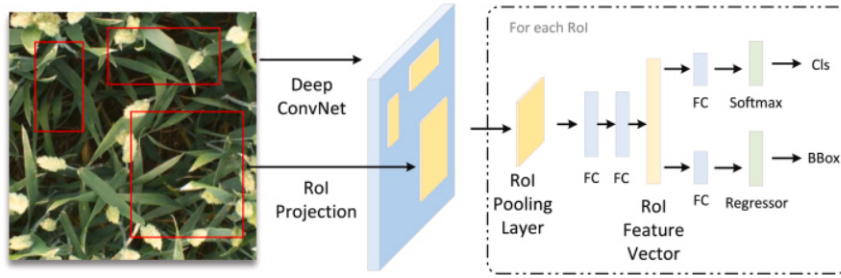
The backbone network mainly includes the *Focus module*, *Convolution-BatchNorm-Swish module (CBS)*, *Spatial Pyramid Pooling (SPP)*, and *CSP Bottleneck with 3 convolutions module (C3)*. The Focus module performs a slicing operation on the input image, expanding the input channel to four times its original size while reducing the width and height of the image to half its original size. The CBS module combines 2D convolution (Conv), batch normalization (BN), and SiLU activation function (SiLU). The C3 module consists of multiple CBS modules that form residual connections.

Moreover, the input feature maps are concatenated with the results of multiple residual connections to form a Cross Stage Partial (CSP) module. This step is to extract the information for fusion on feature maps at different scales to increase the reuse of features.

SPP is a spatial pyramid pooling structure, usually used to solve the problems

of varying input image size and significant variation of object size in images by pooling feature maps of different sizes and stitching the pooled results together.

- **Neck:** After extracting the image features through the backbone network, the backbone and the head are connected using the neck. In this model, the neck combines Feature Pyramid Network (FPN) and Pixel Aggregation Network (PAN) to integrate a regular top-down FPN with a bottom-up PAN to fuse the extracted semantic features with fine-grained positional features. The bottom-up feature extraction network extracts feature maps from the original image at different scales. On the other hand, the top-down feature fusion network is implemented by upsampling and cross-layer connection, which can fuse the low-level feature maps with the high-level feature maps and generate a set of feature maps with different resolutions. This structure can improve the model’s sensitivity to objects of different scales while improving the detection accuracy.
- **Head:** The detection head predicts the feature maps of three scales corresponding to large, medium, and small objects. It detects objects of various sizes and outputs a vector containing the object category probability, confidence, and bounding box position information. Multi-scale feature maps can improve the model’s sensitivity to different scale objects and detection accuracy.



**Fig. 5:** YOLOv5 head mechanism (for more informations see reference [6]).

YOLOv5 also uses many data enhancement techniques. During the training process, YOLOv5 randomly scales the input image, select horizontal flip or vertical flip, randomly crop, and randomly adjust the input image’s brightness, contrast, and saturation. These data augmentation techniques can effectively increase the data’s diversity and improve the model’s generalization ability, thereby improving the model’s performance. The YOLOv5 algorithm retains more features in the feature extraction network and often optimizes the algorithm details. It can combine semantic and detailed information to improve the detection effect of small objects, which leads to the robustness and stability of YOLOv5 in dealing with complex problems.

### 3.2.2 YOLOv8

YOLOv8 represents a further evolution from YOLOv5, introducing several optimizations and architectural improvements (reference [7] on bibliography).

The architecture is divided into three main components: *backbone*, *neck*, and *head*.

The *backbone* is responsible for feature extraction from the input image through convolutional layers and *C2f* blocks. The latter combine convolutions and shortcut mechanisms, enabling better information capture without compromising speed. The *neck*, though not explicitly mentioned in the YOLOv8 configuration file, is described in the official documentation and is responsible for integrating information from various resolution scales. This is done through *upsampling* and concatenation operations, as well as through the *Spatial Pyramid Pooling Fast block (SPF)*, which replaces the previous SPP version with a faster variant.

The *head* manages the final object detection phase, now based on an anchor-free model, further improving inference speed. A key feature of the YOLOv8 architecture is the division of the detection block (*detect*) into three distinct paths, each specialized in detecting objects of different sizes: small, medium, and large. Each path consists of multiple convolutional layers and *concat* operations, which progressively reduce the resolution of feature maps, thus ensuring proper handling of objects at different scales. Another major improvement over YOLOv5 is the introduction of parameters such as depth and width multipliers, which allow for optimal model scalability. The depth multiplier controls the number of bottleneck blocks within the C2f blocks, while the width multiplier and the maximum channel limit adjust the number of channels used during processing. These parameters make YOLOv8 more flexible, allowing the model to be adapted to a wide range of applications, from lighter tasks to more complex ones ([8] on bibliography).

### 3.2.3 YOLOv10

YOLOv10 represents the latest evolution in the YOLO family, introducing significant architectural improvements over its predecessors, aiming to optimize both post-processing and the model architecture itself for object detection (see [9], [10],[11] on bibliography).

It maintains the basic structure of previous YOLO model versions, but with the following improvements:

- **NMS-Free Training:** One of the most significant improvements of YOLOv10 is the elimination of the NMS (Non-Maximum Suppression) post-processing step during training. This was made possible by introducing a consistent dual-label assignment method, allowing the model to make multiple predictions on an object with a confidence score for each. During inference, only the bounding box with the highest confidence score is selected, reducing inference time without compromising accuracy.
- **Holistic Design Optimization:** YOLOv10 has adopted a holistic approach to optimize every component of the model, enhancing efficiency and accuracy. This includes optimizing downsampling layers, basic blocks, and the model head, using

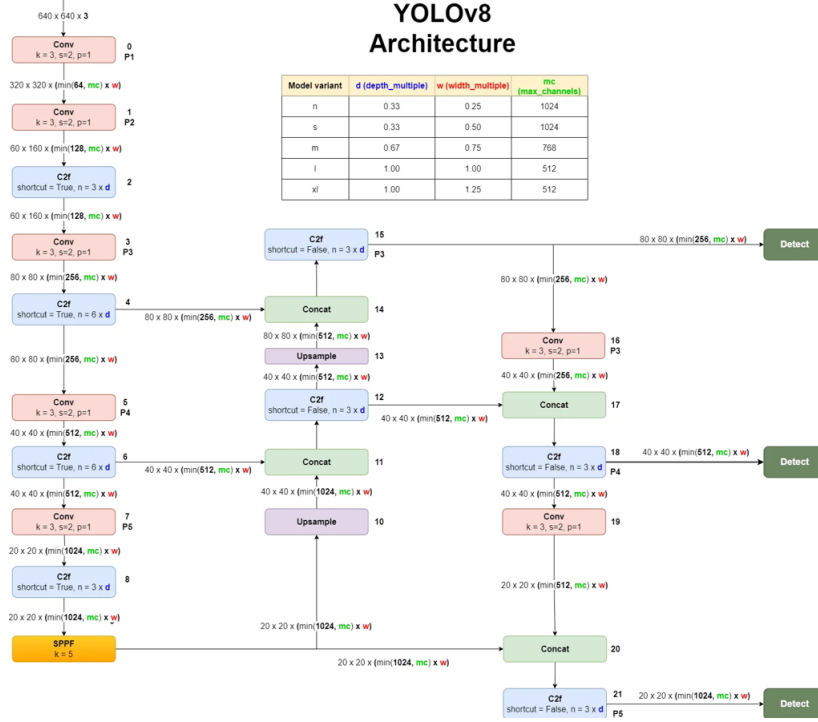


Fig. 6: YOLOv8 Architecture

depth-wise separable convolutions and a rank-guided design to reduce redundancy and improve performance.

- **Partial Self-Attention:** To enhance accuracy without significantly increasing computational costs, YOLOv10 introduces the Partial Self-Attention (PSA) module. This module applies self-attention partially across channels, retaining the benefits of global attention modeling while reducing computational overhead.

### 3.3 Experimental Setup and Analysis Conducted

In our study, our first step was to establish a robust experimental environment by configuring a *Docker container*. This containerization provided a consistent and isolated platform for conducting our experiments, This setup included leveraging the computational capabilities of an **Intel(R) Xeon(R) Gold 6254 CPU @ 3.10GHz** and **GPU type NVIDIA RTX A6000** for our trainings.

Afterward, we prepared our dataset for the training phase. We split the images and their corresponding labels into training (70%), validation (15%), and test (15%) sets, ensuring proper balance and consistency between the images and labels.

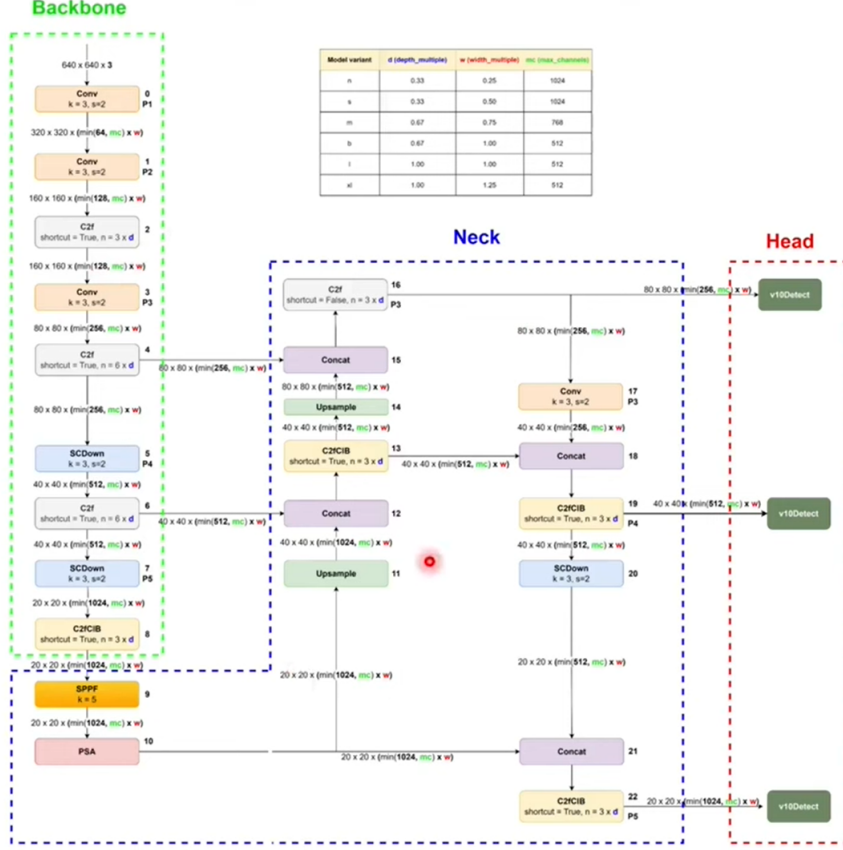


Fig. 7: YOLOv10 Architecture

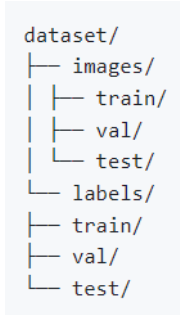


Fig. 8: folders

Additionally, we organized the dataset into the folder structure shown in Figure 8 to streamline the process.

Regarding training phase, within this controlled environment, we systematically performed *hyperparameter tuning* for YOLOv8s and YOLOv10s architectures. This involved iteratively adjusting key parameters to maximize the models' detection accuracy and efficiency.

Upon identifying the optimal configurations, we proceeded to evaluate the models performing inference on some images. We also integrated *Eigen CAM (Class Activation Mapping)* techniques [12] to enhance our understanding of the models' decision-making processes.

Eigen CAM not only facilitated visual interpretation of the detection results but also provided insights into the regions of interest within the detected images. After that, we also evaluated several forms of optimization, the most effective of which

was customizing the network architecture by focusing only on the layers specialized in the detection of small objects.

## 4 Results and Discussion

### 4.1 Hyperparameter Tuning

To achieve maximum effectiveness and efficiency from our training, we performed an hyperparameter tuning. In particular, we performed tests on the following parameters: *number of epochs*, *batch size*, *optimizer*, *learning rate*, *momentum*, *weight decay*, *warmup*, as from Ultralytics documentation, they were identified as some of the most important.

The *batch size* of a neural network refers to the number of data samples processed before the model updates its weights during training. It balances the trade-off between computational efficiency and training stability. Finding the right batch size is crucial for efficient training, balancing accuracy, computational resources, and the model's ability to generalize to new data.

As regards the *optimizer*, it's essential to find the best one for training a neural network because different optimizers can affect convergence speed, training stability, and the model's generalization ability. Choosing the appropriate optimizer can significantly enhance the model's performance in terms of accuracy and efficiency during both training and deployment. In detail, we tested *SGD* and *AdamW* optimizers.

*SGD (Stochastic Gradient Descent)* is a fundamental optimizer for training neural networks. It minimizes the loss function by iteratively updating the model weights in the direction opposite to the gradient of the loss function with respect to the weights. Finding the right learning rate and momentum is crucial to balance model convergence and training speed.

*AdamW* is an enhanced variant of Adam (Adaptive Moment Estimation) that incorporates weight decay regularization directly into the weight updates. It helps manage regularization during training, improving the model's ability to generalize to unseen data.

*Training epochs* represent the number of times the supervised learning algorithm passes through the entire training dataset.

Properly adjusting the number of epochs is crucial for training neural networks in order to avoid underfitting and overfitting errors, and to balance the model training, enabling it to generalize well to new, unseen data.

The *learning rate* is a hyper parameter that controls the size of steps taken during model training, and how aggressively the model learns the data during training. An high learning rate can lead to overshooting the optimal solution. On the other hand, a low learning rate can cause optimization to be slow. For this reason, tuning it correctly is crucial to ensure efficient training and better model performance, because if it's too high, the model may not converge, and if it's too low, training will be slow and may result in poor performance.

*Weight Decay* helps in preventing the model from fitting the training data too closely, reducing overfitting and encouraging better generalization to new data. It acts as a

penalty on the magnitude of the model’s weights. During training, in addition to the original loss function, a penalty term proportional to the sum of the squares of the weights is added. A small weight decay value can cause the model to be prone to overfitting.

The hyper parameter *warmup* refers to the initial warmup process that takes place during model training. The purpose of a warm up is to help the model overcome initial problems in training, such as confusion or unsteadiness when starting from a random starting weight. A small warmup can cause the model not to have enough time to adjust to the training data. On the other hand, a high warmup causes the model to focus too much on the initial training data, which results in overfitting.

*Momentum* is a parameter function as a moving average for the gradient computed in previous steps. It adds a fraction of the previous update to the current update. This allows the optimization process to maintain a consistent direction even when the gradient fluctuates. Momentum values are commonly used range from 0 to 1. 0 indicates that no momentum has been applied. 1 means the previous update is fully accounted for in the current update.

In detail, Table 1 presents the values that were tested for each parameter across the configurations.

| Parameter        | Tested Values |        |        |
|------------------|---------------|--------|--------|
| batch size       | 32            | 64     | 128    |
| optimizer        | SGD           | AdamW  |        |
| number of epochs | 80            | 200    | 300    |
| learning rate    | 0.01          | 0.001  | 0.0001 |
| momentum         | 0.9           | 0.8    | 0.95   |
| weight decay     | 0.0001        | 0.0005 | 0.001  |
| warmup           | 1             | 3      | 5      |

**Table 1:** Tested values for each parameter

Our results show that, among all the possible combinations of parameters, for both YOLOv8s and YOLOv10s architectures, the best configuration is the following:

( epochs: 200, batch size: 64, optimizer: SGD, lr: 0.010, wd: 0.0010, wu: 3, mom: 0.9)

The results given by both the architectures setting this configurations are shown on Table 2. The table highlight the comparative performance of YOLOv8 and YOLOv10

**Table 2:** Best configuration results

| Metrics        | YOLOv8s       | YOLOv10s |
|----------------|---------------|----------|
| mAP50(B)       | <b>0.9802</b> | 0.9754   |
| mAP50-95(B)    | <b>0.7403</b> | 0.6956   |
| precision(B)   | <b>0.9635</b> | 0.9497   |
| recall(B)      | <b>0.9560</b> | 0.9443   |
| train box_loss | <b>0.6593</b> | 0.8908   |

across several metrics. YOLOv8 consistently outperforms YOLOv10 in terms of mean Average Precision (mAP), achieving a mAP50 of 0.9802 compared to YOLOv10’s 0.9754, and a mAP50-95 of 0.7403 versus 0.6956 for YOLOv10, representing a notable improvement of approximately 6.44%. Additionally, YOLOv8 demonstrates superior precision and recall, with values of 0.9635 and 0.9560 respectively, compared to YOLOv10’s precision of 0.9497 and recall of 0.9443.

Most notably, YOLOv8 shows a significant reduction in training box loss, with a value of 0.6593, compared to YOLOv10’s 0.8908. This results in a substantial improvement of about 26.02%, suggesting that YOLOv8 is better at minimizing localization errors during the training phase.

Our results already show significant improvements over the data presented on Figure 3 given by experiments conducted on the same dataset we used for our trainings. In the following Table 3, we aim to compare our results, with the results obtained with Faster R-CNN, RetinaNet, Modified SSD-512 obtained by the SAR experts [5].

**Table 3:** Comparison between all methods developed with our dataset

| Method           | mAP(%)       |
|------------------|--------------|
| Faster R-CNN     | 0.9497       |
| RetinaNet        | 0.9443       |
| Modified SSD-512 | 89.07        |
| <b>YOLOv8s</b>   | <b>98.02</b> |
| <b>YOLOv10s</b>  | <b>97.54</b> |

As shown on the table, our YOLOv8s shows significant improvements, with increases of about 3% in mAP compared to the others, proving to be the most effective method among those developed.

Indeed, other methods, like Faster R-CNN and RetinaNet, present mAP values lower than those of YOLOv8s. As explained before, YOLOv10s, although close in performance with a mAP of 97.54%, remains slightly below YOLOv8s, but still better than the other architectures shown on the table.

Regarding the state-of-the-art YOLOv5 architecture studied for ship detection task [3], we can only make approximate comparisons with our results due to the use of different datasets, specifically *High-Resolution Ship Image Dataset (HRSID)* and *Ship Surveillance with SAR Imaging Dataset (SSID)*, on which we have not tested our architectures due to time constraints. Specifically, the *Ship Surveillance with SAR Imaging Dataset (SSID)* is the first open dataset for SAR ship detection and includes 1,160 images with 2,456 ship targets. The *High-Resolution Ship Image Dataset (HRSID)* comprises images from Sentinel-1 and TerraSAR-X. The experimental results with the SSID dataset demonstrated the highest evaluation metrics when incorporating attention modules and modifying the feature fusion module, the solution proposed by the experts. Additionally, when compared to other methods on the HRSID dataset, such as Faster R-CNN, RetinaNet, and the original YOLOv5

model, the method proposed in this paper achieved a 2.85% increase in mAP. However, keeping in mind that the datasets are different, the types of SAR images used are the same, and making a comparison among the obtained metrics, we can confirm the quality of our results, as shown in the following Table 4.

YOLOv8s achieved a mAP of 98.02%, which represents a significant improvement

**Table 4:** Comparison among results obtained by YOLO series

| Method                  | mAP(%)       | Precision(%) | Recall(%)    |
|-------------------------|--------------|--------------|--------------|
| Improved YOLOv5 (SSID)  | 95.02        | 88.27        | 93.41        |
| Improved YOLOv5 (HRSID) | 85.11        | 84.37        | 84.56        |
| <b>YOLOv8s</b>          | <b>98.02</b> | <b>96.35</b> | <b>95.60</b> |
| <b>YOLOv10s</b>         | <b>97.54</b> | <b>94.97</b> | <b>94.43</b> |

over the results obtained with YOLOv5, which achieved a mAP of 85.11% on the HRSID dataset and 95.02% on the SSID dataset. In terms of precision, YOLOv8s reached 96.35%, significantly surpassing the Precision of YOLOv5 (SSID) architecture. Additionally, in Recall, YOLOv8s excels with a 95.60%, compared to 93.41% achieved by YOLOv5 (SSID).

## 4.2 Evaluating CO2 Emissions

In addition to evaluating performance metrics, it is important to assess the CO2 emissions generated during the training of neural networks to better understand and manage the environmental impact of these computational processes. For this purpose, we employed the tool *CodeCarbon* (reference [13] in the bibliography), which estimates the carbon emissions produced by monitoring the energy consumption of the hardware used, such as CPUs and GPUs. This energy consumption is then converted into an estimate of CO2 emissions based on the energy mix of the location where the computation is performed.

Monitoring CO2 emissions is crucial for several reasons. First, training large neural networks can require substantial amounts of energy, especially when dealing with advanced architectures such as YOLOv8s and YOLOv10s. By estimating and reducing CO2 emissions, we can minimize the environmental footprint of these operations. With the growing demand for artificial intelligence and machine learning, the associated energy consumption is increasing rapidly. Therefore, keeping track of and optimizing energy usage is essential for promoting more sustainable AI development practices. Another relevant aspect is cost efficiency. High energy consumption not only impacts the environment but also increases operational costs. Being aware of energy use allows organizations to implement strategies to reduce both costs and emissions.

Finally, many organizations are committed to reducing their carbon footprint as part of their corporate social responsibility goals. Tools like *CodeCarbon* help monitor progress towards these goals and ensure compliance with increasingly stringent carbon emission regulations.

In Table 5 below, the detailed values of CO2 emissions (in kg) generated during the training session with our best configuration are presented, for both YOLOv8s and

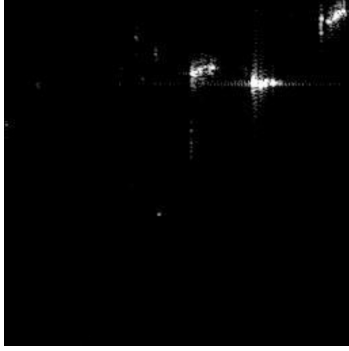
YOLOv10s.

**Table 5:** CO2 Emissions given by YOLOv8s and YOLOv10s trainings

| Architecture | Emissions (Kg) |
|--------------|----------------|
| YOLOv8s      | <b>1.82</b>    |
| YOLOv10s     | 2.23           |

To contextualize the emissions generated, the CO2 produced during model training, while seemingly modest, can accumulate quickly at scale. For instance, the training of the YOLOv8s architecture resulted in 1.8210 kg of CO2 emissions, whereas the YOLOv10s architecture produced 2.2259 kg. In practical terms, 1.8 kg of CO2 is equivalent to driving a typical gasoline-powered car for around 7.5 km. While the emissions from one training session may appear small, the impact becomes significant when multiple training iterations are considered across the industry.

### 4.3 Inference Evaluation



**Fig. 9:** Inference sample

After finding the best parameter configuration for training, we proceeded to the inference phase. The inference evaluation is a critical step in evaluating the real performance of deep learning models, as the predictions are performed on unseen data. We tested the inference on both YOLOv8 and YOLOv10 using a sample of a series of images with the corresponding ground truth provided by Global Fishing Watch (Figure 9). On Tables 6 and 7 there is a summary of the results of the inference process on the image, showing the detected ships (objects), confidence scores, and various representations of their bounding box coordinates, using respectively YOLOv8 and YOLOv10.

Both models were applied to detect the three distinct ships on the sample image (Figure 9), with their confidence scores and bounding box coordinates reported in different formats (xyxy, xyxyn, xywh, xywhn). The bounding box coordinates provide detailed information about the positioning and size of each ship in the image. In particular, the normalized bounding boxes (xyxyn and xywhn) ensure scalability and adaptability across images of different sizes and resolutions.

For YOLOv8, the confidence scores for the detected ships ranged from 0.7836 to 0.8278, indicating a reliable detection. In comparison, YOLOv10 achieved higher confidence scores, ranging from 0.8390 to 0.8603, which may suggest slightly better detection accuracy. However, the bounding box values across the two models show consistency,

**Table 6:** Inference results given by YOLOvs8

| Metric                      | Ship 1                                   | Ship 2                                   | Ship 3                                |
|-----------------------------|------------------------------------------|------------------------------------------|---------------------------------------|
| <b>Confidence</b>           | 0.8278                                   | 0.7923                                   | 0.7836                                |
| <b>Bounding Box (xyxy)</b>  | [337.6080, 109.4396, 392.3931, 145.9810] | [447.0654, 133.7483, 508.8826, 169.1829] | [588.7632, 9.5357, 633.0937, 48.7354] |
| <b>Bounding Box (xyxyn)</b> | [0.5275, 0.1710, 0.6131, 0.2281]         | [0.6985, 0.2090, 0.7951, 0.2643]         | [0.9199, 0.0149, 0.9892, 0.0761]      |
| <b>Bounding Box (xywh)</b>  | [365.0006, 127.7103, 54.7851, 36.5413]   | [477.9740, 151.4656, 61.8173, 35.4345]   | [610.9285, 29.1356, 44.3304, 39.1997] |
| <b>Bounding Box (xywhn)</b> | [0.5703, 0.1995, 0.0856, 0.0571]         | [0.7468, 0.2367, 0.0966, 0.0554]         | [0.9546, 0.0455, 0.0693, 0.0612]      |

**Table 7:** Inference results given by YOLOvs10

| Metric                      | Ship 1                                 | Ship 2                                | Ship 3                                 |
|-----------------------------|----------------------------------------|---------------------------------------|----------------------------------------|
| <b>Confidence</b>           | 0.8603                                 | 0.8593                                | 0.8390                                 |
| <b>Bounding Box (xyxy)</b>  | [134.7285, 44.1035, 156.5727, 58.5263] | [234.8712, 3.3276, 253.1622, 19.5238] | [178.6259, 53.3526, 203.9755, 68.0978] |
| <b>Bounding Box (xyxyn)</b> | [0.5263, 0.1723, 0.6116, 0.2286]       | [0.9175, 0.0130, 0.9889, 0.0763]      | [0.6978, 0.2084, 0.7968, 0.2660]       |
| <b>Bounding Box (xywh)</b>  | [145.6506, 51.3149, 21.8442, 14.4228]  | [244.0167, 11.4257, 18.2910, 16.1961] | [191.3007, 60.7252, 25.3497, 14.7452]  |
| <b>Bounding Box (xywhn)</b> | [0.5689, 0.2004, 0.0853, 0.0563]       | [0.9532, 0.0446, 0.0714, 0.0633]      | [0.7473, 0.2372, 0.0990, 0.0576]       |

with YOLOv10 providing similar or smaller bounding boxes in both absolute and normalized forms. This might imply tighter localization of the detected objects.

#### 4.3.1 Explainability

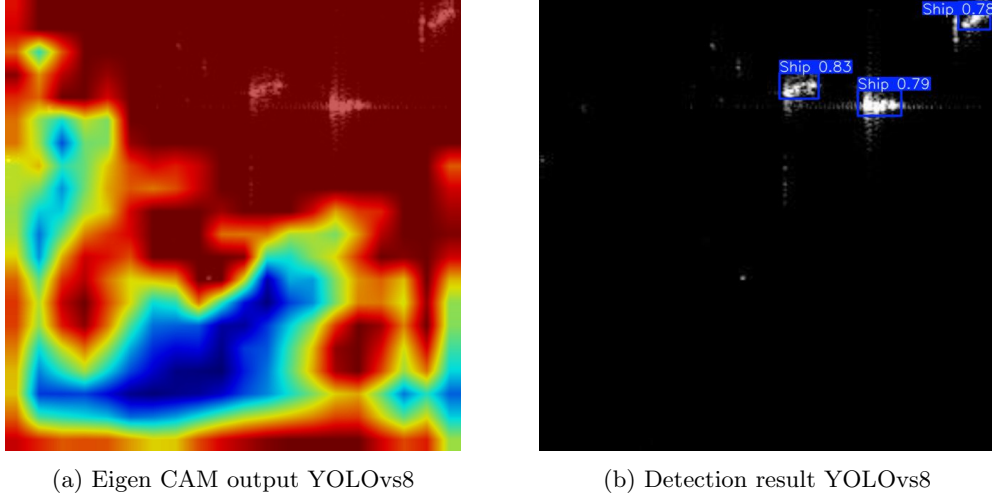
To visualize the image regions that have significantly contributed to the models’ decisions during the inference process, and understand better their behavior, we used *Eigen CAM* (see [12] on bibliography).

*CAM (Class Activation Mapping)* is a technique applied in neural networks to generate an activation map that highlights the regions of an image that have most influenced the model’s prediction.

*Eigen CAM* is a variant of *CAM* that incorporates the concepts of eigenvalues and eigenvectors to improve the stability and interpretation of activation maps. Specifically, it utilizes eigenvalue analysis of the activation matrix to refine the generated activation map.

In Figure 10 and Figure 11 below, the output generated by Eigen CAM on the image processed by our models are shown. Given the input image (Figure 9), as it’s possible to observe in Figures 10a and 11a, Eigen CAM highlights regions in blue that were deemed less important, while areas in red are those that had a greater influence on the inference process. On the right, in Figure 10b and Figure 11b, there are the graphical results of the inferences, with bounding boxes and their corresponding confidence scores.

Thanks to the application of EigenCam, we can also confirm the results observed before in Tables 6 and 7. Indeed, while YOLOv8s shows good performance and a strong ability to focus on areas of interest (see Figure 10a, YOLOv10 can isolate and narrow down these areas even further, demonstrating greater precision and a stronger ability to distinguish objects in the images (Figure 11a).

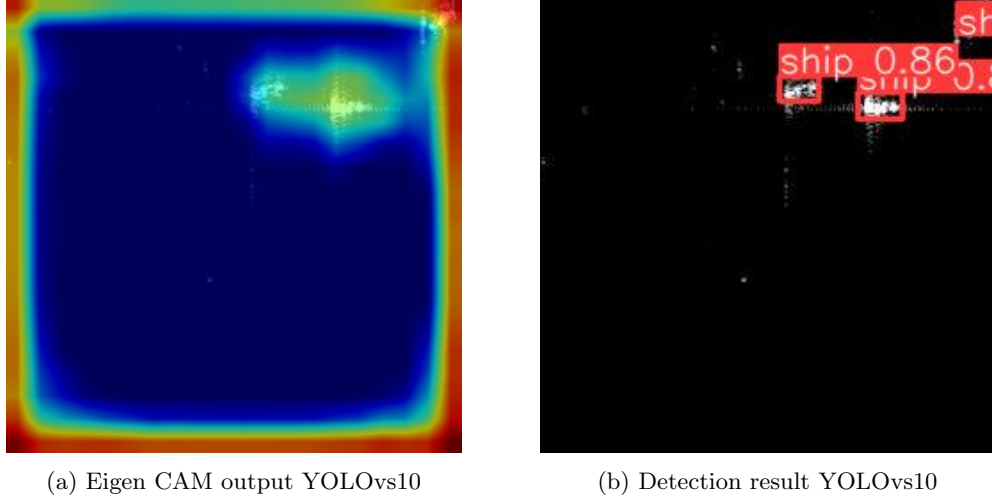


**Fig. 10:** Inference process visual results on yolov8s

## 5 Optimization

The final part of our work was dedicated to an optimization phase. We aimed to enhance the network architecture specifically for small object detection, tailoring it more closely to our task. To achieve this, we employed a layer-based pruning approach. This technique involves systematically removing certain layers from the network architecture to streamline the model, in order to reduce the network’s complexity and computational load. The primary objective of this optimization was to increase the efficiency of the model, making it more suitable for deployment on edge devices. Layer-based pruning helps in creating a more computationally sustainable network, which is crucial for running models on hardware with limited resources, enabling their practical application in environments where computational power is constrained.

After an intense study of the layers that constitute the network architectures, we then moved to the practical phase by directly removing layers from the architecture file. Indeed, this file describes the various layers that form the backbone and head, arranged in the same order as they appear in the diagrams presented in Figures 7 and 6. In particular, we could use this strategy because, in both YOLOv8s and YOLOv10s architectures, the detection is divided into three blocks: one for small objects, one for



**Fig. 11:** Inference process visual results on yolovs10

medium objects, and one for large objects.

To go more in-depth into this concept, a detailed explanation of the YOLOv8s optimization process will be provided. The procedure for YOLOv10s is the same, taking into account the slight architectural differences between the two models.

```
# YOLOv8.0n backbone
backbone:
  # [from, repeats, module, args]
  - [-1, 1, Conv, [64, 3, 2]] # 0-P1/2
  - [-1, 1, Conv, [128, 3, 2]] # 1-P2/4
  - [-1, 3, C2f, [128, True]]
  - [-1, 1, Conv, [256, 3, 2]] # 3-P3/8
  - [-1, 6, C2f, [256, True]]
  - [-1, 1, Conv, [512, 3, 2]] # 5-P4/16
  - [-1, 6, C2f, [512, True]]
  # - [-1, 1, Conv, [1024, 3, 2]] # 7-P5/32
  # - [-1, 3, C2f, [1024, True]]
  - [-1, 1, SPPF, [1024, 5]] # 9 -> 7
```

(a) Removed layers on backbone

```
# YOLOv8.0n head
head:
  # - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
  # - [[-1, 6], 1, Concat, [1]] # cat backbone P4
  # - [-1, 3, C2f, [512]] # 12

  - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
  - [[-1, 4], 1, Concat, [1]] # cat backbone P3
  - [-1, 3, C2f, [256]] # 15 (P3/8-small) -> 10

  # - [-1, 1, Conv, [256, 3, 2]]
  # - [[-1, 12], 1, Concat, [1]] # cat head P4
  # - [-1, 3, C2f, [512]] # 18 (P4/16-medium)

  # - [-1, 1, Conv, [512, 3, 2]]
  # - [[-1, 9], 1, Concat, [1]] # cat head P5
  # - [-1, 3, C2f, [1024]] # 21 (P5/32-large)

  - [[10], 1, Detect, [nc]] # Detect(P3, P4, P5)
  ...
```

(b) Removed layers on head

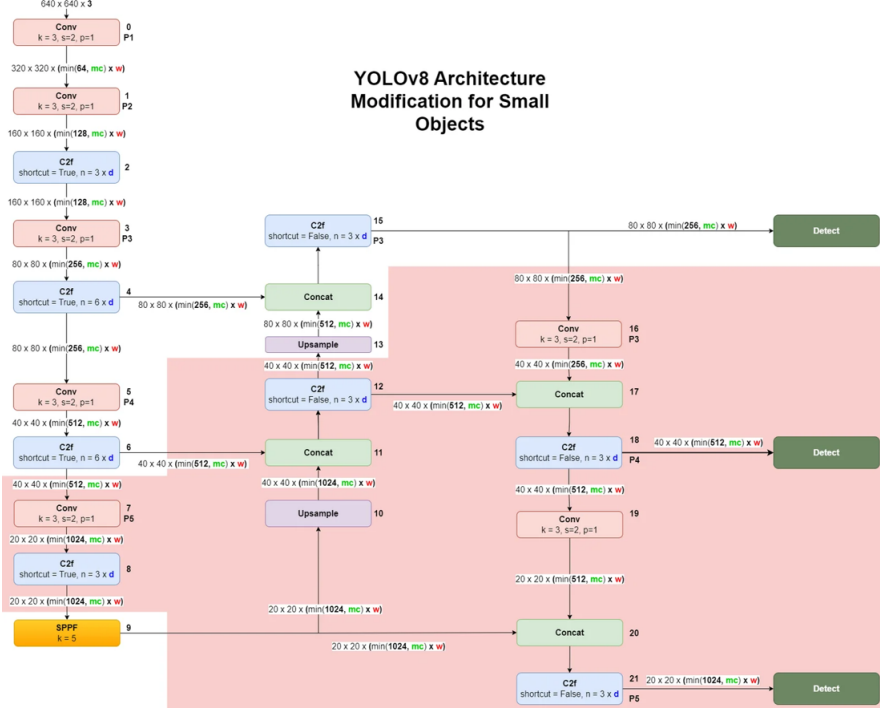
**Fig. 12:** yolov8.yaml

To optimize the YOLOv8 model for small object detection, we made specific modifications to the original YOLOv8 architecture by removing certain layers that handle

the detection of medium and large objects.

In the architecture configuration file (see `yolov8.yaml` on Figure 12), we commented out the deeper layers responsible for detecting objects at higher scales. The removed layers include those that extract features from larger receptive fields, which are less relevant for detecting smaller objects.

As shown in Figure 13, we eliminated the Conv and C2f layers associated with the P5/32 path (originally present at layers 7 and 8 in the backbone, see Figure 12a) and the corresponding parts of the head that aggregate features from the larger scales (e.g., the Concat and C2f layers for P4/16 and P5/32 in the head, see Figure 12b).



**Fig. 13:** Modified YOLOv8 architecture for small objects, with removed layers highlighted.

By removing these layers, we simplified the model to focus solely on detecting objects at the P3 scale, which corresponds to smaller object detection. This modification reduces computational overhead while improving the model's efficiency for small object detection tasks. The final architecture retains only the layers necessary for handling small objects, making it more specialized and efficient in detecting fine details in images. The final architecture is shown in Figure 14.

Table 8 presents the results of training the new architecture using the best configuration identified at the beginning of the analysis, compared to the results obtained from

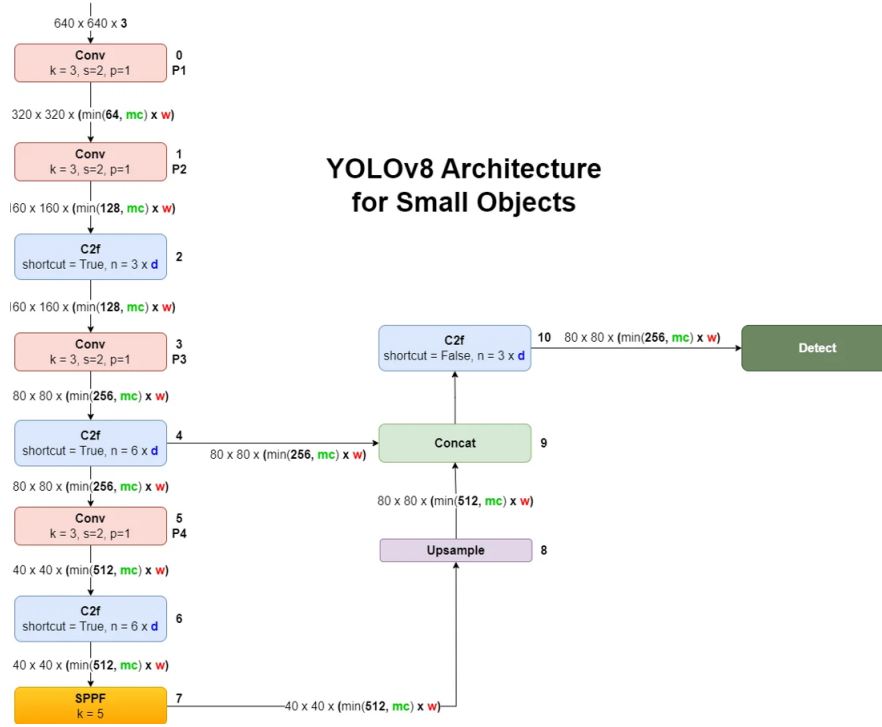


Fig. 14: YOLOv8 architecture for small object detection

the original architecture.

Table 8: Best configuration tested on original YOLOv8 architecture and the another one for small objects

| Metrics        | configuration 2 | configuration 2 (custom) |
|----------------|-----------------|--------------------------|
| runtime (s)    | <b>31106</b>    | <b>25644</b>             |
| mAP50(B)       | 0.9802          | 0.8875                   |
| mAP50-95(B)    | 0.7403          | 0.6949                   |
| precision(B)   | 0.9635          | 0.9468                   |
| recall(B)      | 0.9560          | 0.9402                   |
| train box_loss | 0.65925         | 0.95784                  |

The analysis of the two configurations highlights a significant trade-off between execution time and performance metrics. The **YOLOv8 Small Custom** architecture configuration reduces the training time to **25644** seconds, compared to **31106** seconds for the standard one. This results in a notable time advantage, with a reduction of approximately **18%**, or around **5462** seconds less.

Regarding performance, the small configuration shows a slight decrease in metrics

compared to the standard configuration. The **mAP50-95** is **0.6949** in the small configuration, compared to **0.7403** in the standard configuration, representing a difference of about **6%**. Precision and recall metrics are also slightly lower, with precision at **0.9468** compared to **0.9635**, and recall at **0.9402** compared to **0.9560**. Although there is a decrease, it is minimal, and the performance of the small configuration remains competitive.

So, in summary, the **YOLOv8 Small Custom** configuration offers a clear advantage in terms of **time efficiency**, with a significant reduction in execution times, at the cost of a **minimal penalty in performance**. This trade-off makes it particularly suitable for applications where speed is more crucial than achieving the highest accuracy.

## 6 Final Considerations

In conclusion, the results obtained from our ship detection project using YOLOv8 and YOLOv10 have shown significant improvements in terms of precision and efficiency compared to previous versions. The results indicate that both YOLOv8s and YOLOv10s architectures are highly effective for the task of ship detection, with YOLOv10 showing a slight superiority in overall precision on inference. Furthermore, we managed to find also a trade-off between speed and accuracy of the model through our customized architectures for small objects.

A crucial aspect of our study was also the analysis of CO2 emissions during model training. Using the CodeCarbon tool, we monitored and calculated the emissions, highlighting the importance of considering the environmental impact of neural network training operations and balancing computational efficiency with environmental sustainability.

This project has provided a valuable opportunity to explore and better understand the dynamics of maritime object detection through SAR images, highlighting the importance of combining technological advancements with sustainable practices. Ship detection is crucial for maritime surveillance, traffic management, and security applications, and our results demonstrate that YOLOv8 and YOLOv10 architectures can provide effective and precise solutions.

However, the path is not without challenges. The complexity of SAR images, with their variability in terms of resolution, angle, and weather conditions, poses significant questions for detection accuracy. Our experiments have shown that while advanced technologies like YOLO can significantly improve precision, it is essential to continue developing methods that can better adapt and generalize to different operational conditions.

Furthermore, the aspect of environmental sustainability cannot be ignored. As technology continues to progress, it is essential for the research and development community to consider the ecological impact of its activities. Our analysis of CO2 emissions represents a step in this direction, suggesting that computational efficiency must be balanced with the need to reduce the carbon footprint.

## 7 Future works

Looking ahead, there are several promising directions for future work to build upon the findings of this project.

First of all, considering the possibility of having objects belonging to different classes, rather than just one class as in our case, it would be useful to approach the task not only as a detection task but also as a classification task. This would allow us to distinguish between different objects or types of vessels.

One of the main areas for further research is to explore the integration of different convolutional backbones to enhance the efficiency and accuracy of the detection networks. Specifically, utilizing *MobileNet* as a convolutional backbone could significantly increase the computational efficiency of the network. *MobileNet*, known for its lightweight architecture and high performance in mobile and embedded vision applications, could reduce the computational load and energy consumption while maintaining or even improving detection accuracy.

Finally, continuing to address the environmental impact of model training and deployment remains crucial. Future research should focus on developing energy-efficient training protocols and optimizing the computational resources required for both training and inference. This could include exploring more sustainable hardware options, optimizing hyperparameters for energy efficiency, and utilizing techniques such as model pruning and quantization to reduce the overall carbon footprint of the neural networks.

## References

- [1] Yasir, M., Jianhua, W., Xu, M., Hui, S., Zhe, Z., Shanwei, L., Isiacik Colak, A.T., Hossain, M.S.: Ship detection based on deep learning using sar imagery: a systematic literature review. *Soft Computing* **27**, 1–22 (2022) <https://doi.org/10.1007/s00500-022-07522-w>
- [2] Iervolino, P., Guida, R.: A novel ship detector based on the generalized-likelihood ratio test for sar imagery. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* **PP**, 1–15 (2017) <https://doi.org/10.1109/JSTARS.2017.2692820>
- [3] Yu, C., Shin, Y.: Sar ship detection based on improved yolov5 and bifpn. *ICT Express* **10**(1), 28–33 (2024) <https://doi.org/10.1016/j.ict.2023.03.009>
- [4] Yu, C., Shin, Y.: A deep learning-based sar ship detection. In: 2023 International Conference on Artificial Intelligence in Information and Communication (ICAIIIC), pp. 744–747 (2023). <https://doi.org/10.1109/ICAIIIC57133.2023.10067131>
- [5] Wang, Y., Wang, C., Zhang, H., Dong, Y., Wei, S.: A sar dataset of ship detection for deep learning under complex backgrounds. *Remote Sensing* **11**(7) (2019) <https://doi.org/10.3390/rs11070765>
- [6] Huang, Y., Qian, Y., Wei, H., Lu, Y., Ling, B., Qin, Y.: A survey of deep learning-based object detection methods in crop counting. *Computers and Electronics in Agriculture* **215**, 108425 (2023) <https://doi.org/10.1016/j.compag.2023.108425>
- [7] Jocher, G., Chaurasia, A., Qiu, J.: Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>
- [8] Udemy: YOLOv8 Performance Improvement Masterclass. Accessed: September 2024. <https://www.udemy.com/share/109HNw3@abkL4U7vHokZe2LV4YEmKscPKb952-Wuk0kLPpPDj-xh3SkEyyGmWh0TA1sp47jbeg=/>
- [9] Roboflow: YOLOv8: State-of-the-Art Object Detection Explained. Accessed: September 2024 (2023). <https://youtu.be/A6rHMzRvs98>
- [10] THU-MIG: YOLOv10: Official GitHub Repository. Accessed: September 2024 (2024). <https://github.com/THU-MIG/yolov10>
- [11] Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J., Ding, G.: Yolov10: Real-time end-to-end object detection. *ArXiv abs/2405.14458* (2024)
- [12] S., R.R.: YOLO-V8-CAM. <https://github.com/rigvedrs/YOLO-V8-CAM> (2024)
- [13] Courty, B., Schmidt, V., Luccioni, S., Goyal-Kamal, MarionCoutarel, Feld, B.,

Lecourt, J., LiamConnell, Saboni, A., Inimaz, supatomic, Léval, M., Blanche, L., Cruveiller, A., ouminasara, Zhao, F., Joshi, A., Bogroff, A., Lavoreille, H., Laskaris, N., Abati, E., Blank, D., Wang, Z., Catovic, A., Alencon, M., Bauer, C., Araújo, L.O.N., JPW, MinervaBooks: Mlco2/codecarbon: V2.4.1. <https://doi.org/10.5281/zenodo.11171501> . <https://doi.org/10.5281/zenodo.11171501>